



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



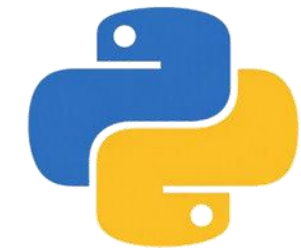
Día, Fecha:	4 de Agosto de 2025
Hora de inicio:	09:00 - 10:40

Lenguajes Formales y De Programación B-

José Estuardo Orellana Leonardo

Iniciamos a las 09:04

Estructura de Datos en Python y Lectura, Escritura de Archivos



Lenguajes Formales y de Programación

Escuela de Ingeniería de Ciencias Y Sistemas

Facultad de Ingeniería

Universidad de San Carlos de Guatemala

Agenda



RECORDATORIOS

Foro 3



Arreglos en Python



Estructuras de datos en Python



Iteracion en Python



Lectura de Archivos



Escritura de Archivos



Lectura Practica

Unica

Recursos

Link del Video



<https://www.youtube.com/watch?v=cwRQHcaef5c>

Dudas clase anterior ?

Tema visto: Buenas prácticas en Python y Funciones en Python



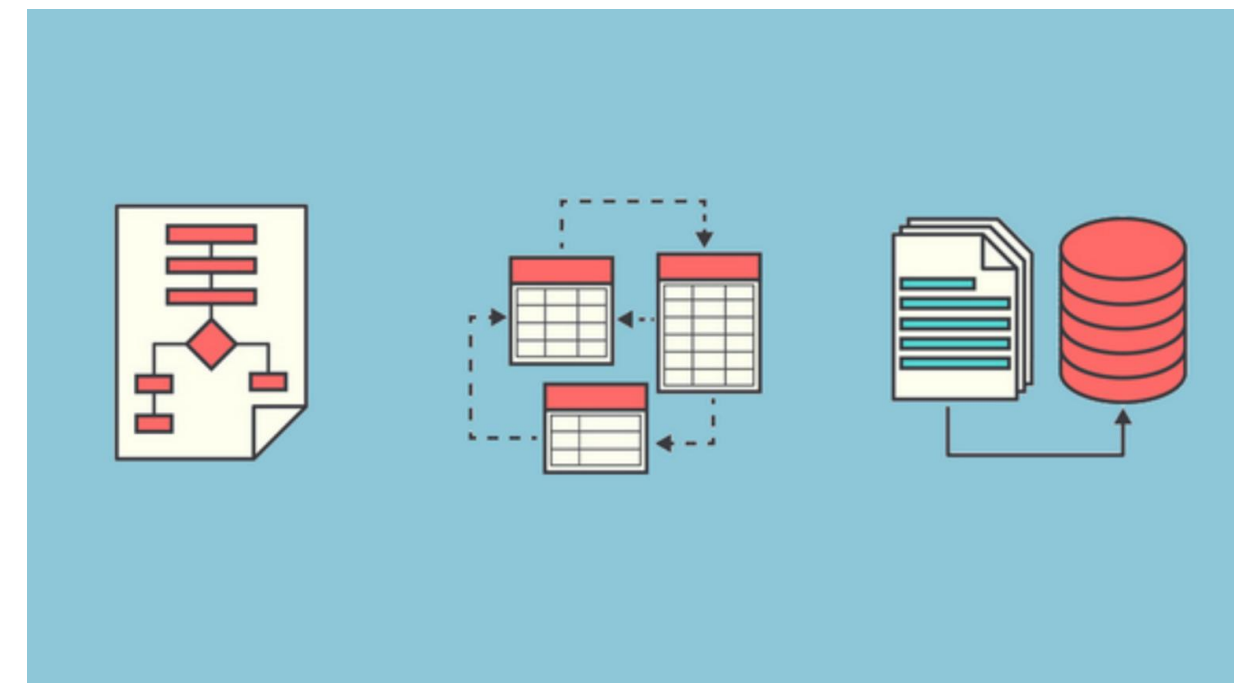
LECTURA PRÁCTICA ÚNICA

**LECTURA Y ESCRITURA
DE ARCHIVOS**



COMPETENCIA QUE DESARROLLAREMOS

El estudiante implemente estructuras de datos en Python y manipule archivos mediante operaciones de lectura y escritura.



Arreglos en Python

En Python, una lista es un tipo de dato que permite almacenar múltiples valores en una sola variable, accesibles a través de un índice numérico. Son fundamentales para manipular colecciones de datos y ofrecen una variedad de métodos para agregar, eliminar, transformar y buscar elementos.

```
1 # Variables con numeros
2 numeros = [0, 1, 2, 3, 4, 5, 6]
3 print(numeros)
4
5 # Variable con frutas (strings)
6 frutas = ["banana", "apple"]
7 print(frutas)
```

Tipos de Listas en Python

1. Listas Unidimensionales:

Son la forma más básica de lista, donde los elementos se organizan en una sola secuencia, accedidos por índices numéricos (0, 1, 2, etc.).

```
1 # Listas unidimensionales
2 numeros = [1, 2, 3, 4, 5]
3 # Acceso a elementos
4 print(numeros[0]) # Imprime 1
```

Tipos de Listas en Python

2. Listas Bidimensionales:

Son listas que contienen otras listas como elementos, creando una estructura de datos más compleja, como matrices.

```
1 # Listas multidimensionales
2 matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
3 # Acceder a un elemento específico
4 print(matriz[1][2]) # Imprime 6
```

Tipos de Listas en Python

3. Arreglos Asociativos (Diccionarios):

En Python, los arreglos asociativos se implementan con diccionarios. En lugar de usar índices numéricos, se utilizan claves para acceder a los elementos.

```
1 # Arreglos asociativos (diccionarios)
2 persona = {
3     "nombre": "Juan",
4     "edad": 30,
5     "profesion": "Ingeniero"
6 }
7 print(persona["nombre"]) # Juan
```

ACCESO A LOS ELEMENTOS DE UNA LISTA

- Acceso: Se usan índices para obtener el valor de una posición de la lista.
- Índices: Comienzan desde 0. Para una lista de longitud N, sus índices van desde 0 hasta N - 1.

```
# Acceso lista unidimensional
lista_uni = [1, 2, 3, 4, 5]
print(lista_uni[0]) # Primer elemento

# Acceso lista bidimensional
lista_bi = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(lista_bi[1][2]) # Fila 2, elemento 3
```

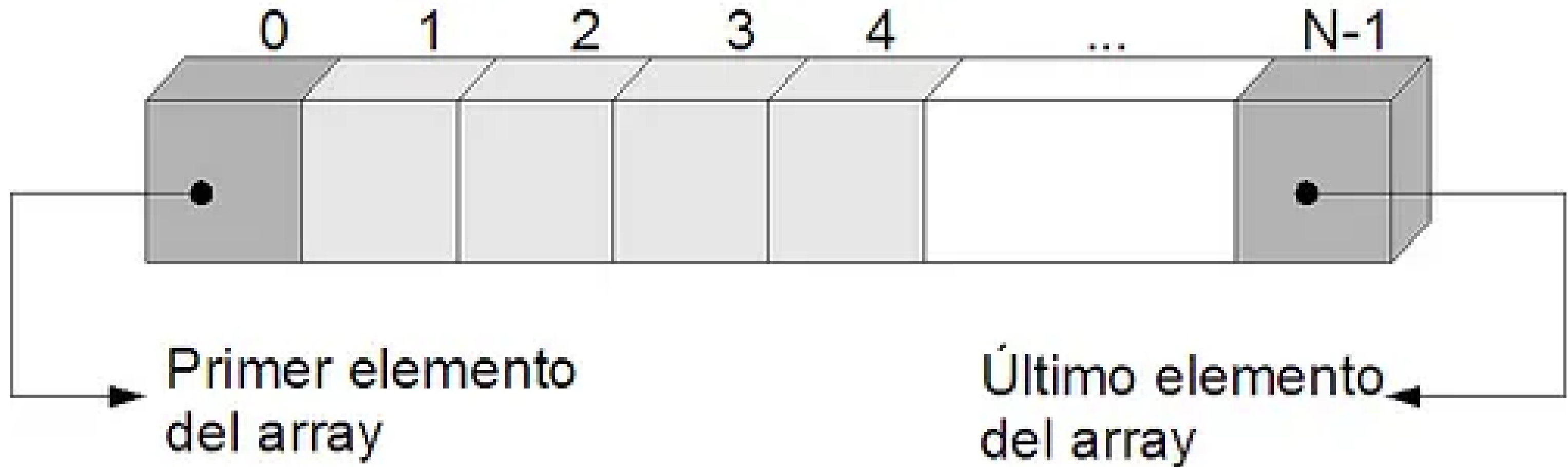
MODIFICACIÓN DE LOS ELEMENTOS DE UNA LISTA

- Al igual que para el acceso se usan índices para reasignar el valor de una posición.

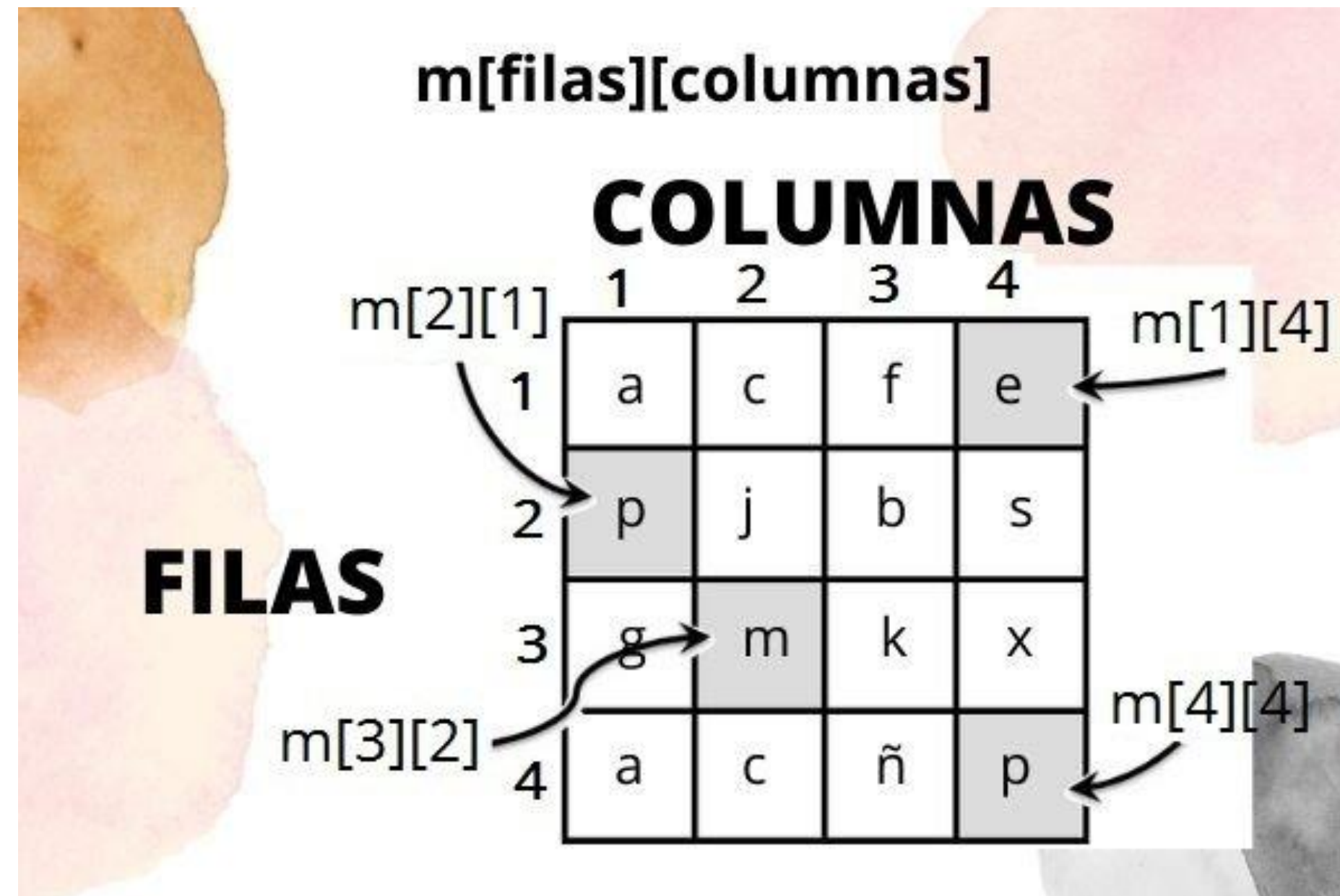
```
# Modificación de una lista unidimensional
lista_uni = [1, 2, 3, 4, 5]
lista_uni[2] = 10 # Cambia el 3er elemento a 10

# Modificación de una lista bidimensional
lista_bi = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
lista_bi[1][2] = 12 # Cambia elem [1][2] a 12
print(lista_bi)
```

REPRESENTACIÓN GRÁFICA DE UNA LISTA UNIDIMENSIONAL



REPRESENTACIÓN GRÁFICA DE UNA LISTA BIDIMENSIONAL



ESTRUCTURAS DE DATOS EN PYTHON



El diccionario (dict) es una colección de pares clave-valor que mantiene el orden de inserción y permite usar **cualquier tipo de dato inmutable como clave**.

```
1 # Estructura de datos dict en Python
2 users = {}
3 # Agregar elementos al diccionario
4 #           Clave           Valor
5 users["Juan"] = {"edad": 12, "rol": "admin"}
6 users["Maria"] = {"edad": 25, "rol": "cliente"}
7 print(users["Juan"]["rol"])    # Imprime: admin
8 print(users["Maria"]["edad"])  # Imprime: 25
9
10 # users[clave] = valor -> Agrega un elemento
11 # users[clave]         -> Obtiene un elemento
```

TUPLAS EN PYTHON



Una tupla (tuple) es una colección ordenada de elementos definida con paréntesis (). Su característica principal es que es **inmutable**: una vez creada, no se puede modificar. Es ideal para datos que no deben cambiar.

```
# Crear una tupla
punto = (10, 20)
colores = ("rojo", "verde", "azul")

# Acceso por índice (inicia en 0)
print(colores[0])    # rojo
print(colores[-1])   # azul

# Inmutable: la siguiente línea daría error
# colores[0] = "negro"
```

TUPLAS: CARACTERÍSTICAS Y USOS



Las tuplas son ordenadas, indexables y pueden anidarse. Al ser inmutables son más rápidas que las listas y pueden usarse como **claves de un diccionario**. También sirven para agrupar y retornar varios valores a la vez.

```
# Desempaquetado (unpacking)
x, y = (10, 20)

# Recorrer una tupla
for color in ("rojo", "verde"):
    print(color)

# Retornar varios valores a la vez
def dividir(a, b):
    return (a // b, a % b)
```

ITERACIÓN CLÁSICA

Iteración utilizando estructuras cíclicas.

- for
- while
- while con break (equivale a do-while)

```
1 for i in range(10):
2     print(i)
3
4
5 i = 0
6 while i < 10:
7     print(i)
8     i += 1
9
10
11 # Equivalente a do-while
12 i = 0
13 while True:
14     print(i)
15     i += 1
16     if not (i < 10):
17         break
```

ITERACIONES MÁS USADAS

for-in

```
1 nombres = ["River", "Ana", "Luis"]
2
3 for nombre in nombres:
4     print(f"Hola, {nombre}")
5
6 # Salida:
7 # Hola, River
8 # Hola, Ana
9 # Hola, Luis
```

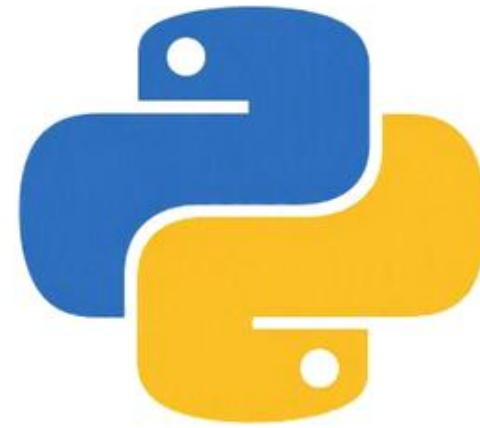
for con range()

```
1 nombres = ["River", "Ana", "Luis"]
2
3 for i in range(len(nombres)):
4     print(f"Hola, {nombres[i]}")
5
6 # Salida esperada:
7 # Hola, River
8 # Hola, Ana
9 # Hola, Luis
```

PYTHON



Python es un lenguaje de programación interpretado, de alto nivel, de código abierto y multiplataforma, construido con un enfoque en la legibilidad del código. Permite a los desarrolladores escribir programas claros y concisos, tanto para tareas sencillas como para sistemas complejos.



En esencia, Python es un lenguaje versátil que se utiliza para el desarrollo web (backend), la ciencia de datos, la automatización, la inteligencia artificial y las aplicaciones de escritorio.

Gestores de paquetes de Python



pip

Es el gestor de paquetes predeterminado de Python y el más utilizado en el ecosistema.

conda

conda

Un gestor de paquetes y entornos muy usado en ciencia de datos, capaz de manejar dependencias más allá de Python.

poetry

poetry

Una herramienta moderna que gestiona dependencias y entornos virtuales priorizando la reproducibilidad del proyecto.

Dependencias en Python

En Python, las dependencias son paquetes o bibliotecas externas que tu proyecto necesita para funcionar correctamente. Estas dependencias se especifican en el archivo requirements.txt y se instalan utilizando el administrador de paquetes pip (Package Installer for Python).

Las dependencias de producción son esenciales para el funcionamiento de la aplicación, mientras que las dependencias de desarrollo son herramientas y bibliotecas utilizadas durante el desarrollo pero no necesarias en producción.



ENTORNOS VIRTUALES (venv)



Un entorno virtual (venv) es un directorio aislado con su propia instalación de Python y sus paquetes. Permite que cada proyecto tenga sus **propias dependencias** sin afectar a otros proyectos ni al sistema, evitando conflictos entre versiones.

```
# 1. Crear el entorno virtual
python -m venv venv

# 2. Activar el entorno
venv\Scripts\activate      # Windows
source venv/bin/activate   # Linux / macOS

# 3. Instalar y registrar paquetes
pip install requests
pip freeze > requirements.txt

# 4. Salir del entorno
deactivate
```

LECTURA DE ARCHIVOS

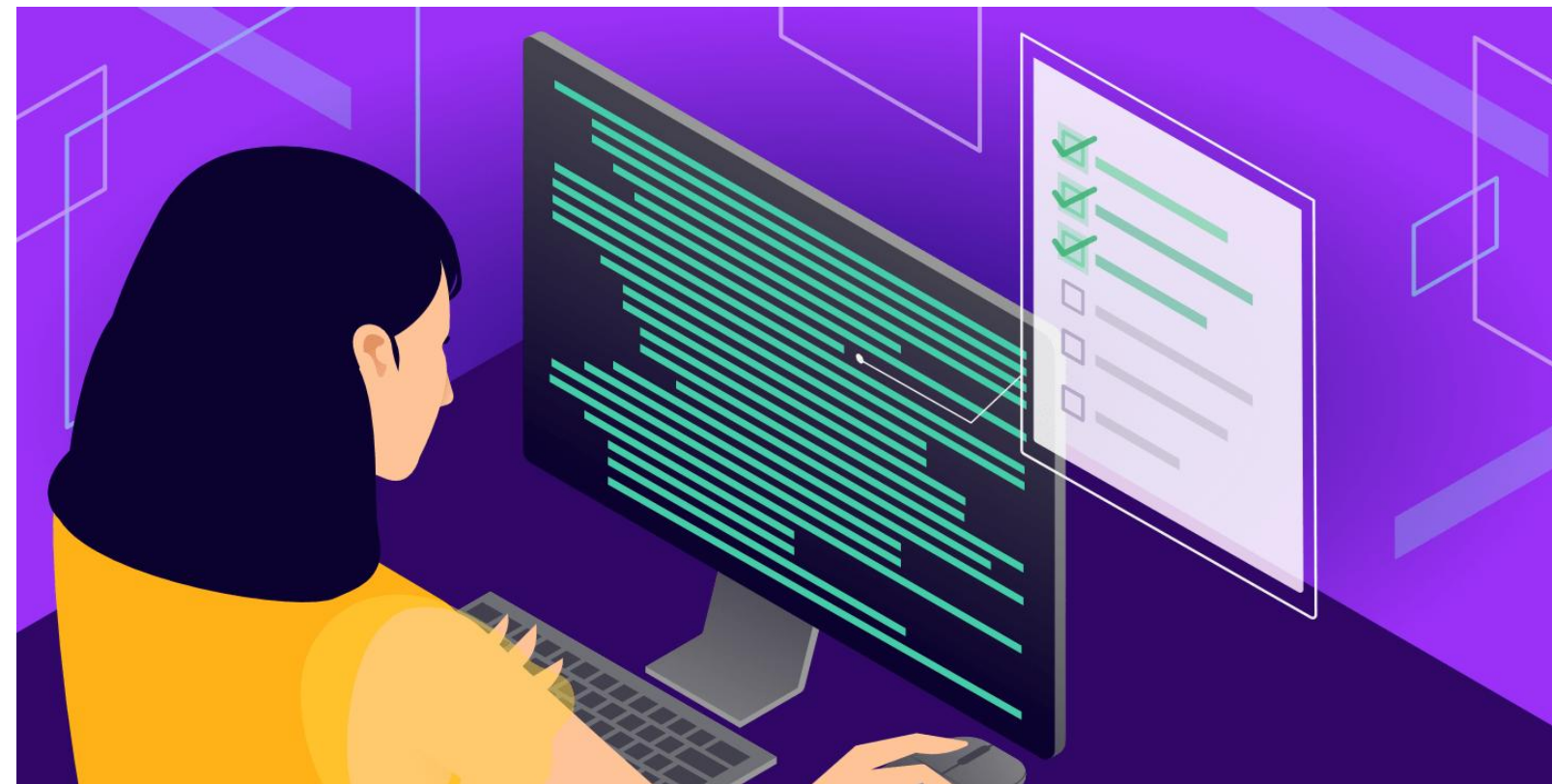


La **lectura de archivos** es el proceso mediante el cual un programa accede al contenido de un archivo almacenado en el sistema y lo carga en memoria para poder procesarlo o mostrarlo.

En Python, esto se realiza con la función nativa **open()**, que permite leer archivos de forma **completa** (con `read()`) o **línea por línea** (iterando el archivo o con `readline()`).

```
1 file_path = "./example.txt"
2
3 # Lectura completa del archivo
4 with open(file_path, "r",
5           encoding="utf-8") as f:
6     data = f.read()
7     print(data)
8
9 # Con manejo de errores
10 try:
11     with open(file_path) as f:
12         print(f.read())
13 except OSError as err:
14     print("Error:", err)
```

EJEMPLO PRÁCTICO



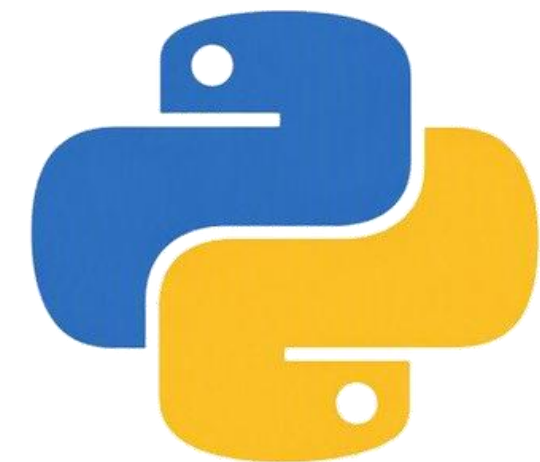
CONCEPTOS CLAVE APRENDIDOS

Listas en Python

- Estructura de datos dinámica
- Acceso a elementos por índice
- Se recorre con bucles (*for*, *for-in*)

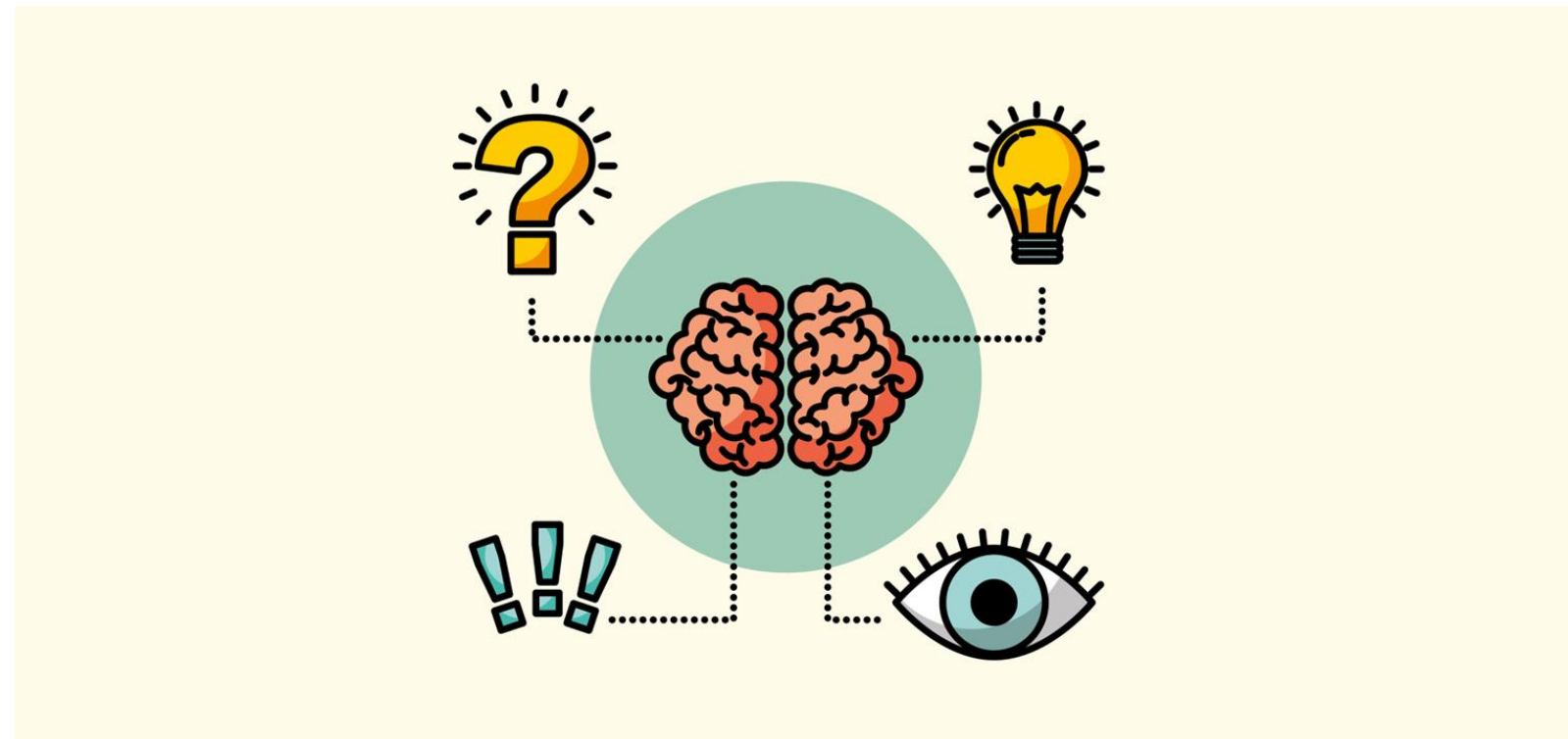
Estructuras de Datos en Python

- Uso de colecciones dinámicas dict
- Iteración clásica *for*, *while*
- Qué es Python
- Lectura de Archivos



VALORES

Pensamiento Lógico: Es necesario razonar paso a paso el proceso completo del funcionamiento de un programa y cómo se almacenan los datos.



REFERENCIAS

- <https://docs.python.org/es/3/tutorial/datastructures.html>
- <https://docs.python.org/es/3/tutorial/inputoutput.html>
- <https://devdocs.io/python/>



**! GRACIAS POR SU ATENCIÓN DUDA!
DUDAS**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA